Recommender Systems - Course Project Report

Part 1 version: data, protocol and baselines

Team	Group 2
Members	Anas Uddin (153215593), Melek Chelik (153663543), Menaka Wellane Hewage
For Part 3 track: select one	Not chosen yet (announced with Part 2)
GitHub repository link	https://github.com/theanasuddin/movielens-recsys

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

MovieLens 1M has 1,000,209 ratings by 6,040 users of 3,706 movies (177 of the 3,883 titles in movies.dat were never rated), so 4.47% of the user-movie matrix is filled. Every user has at least 20 ratings (median 96, maximum 2,314); movies have between 1 and 3,428 (median 123.5; 114 movies have one rating). Ratings lean positive: 1 to 5 stars take 5.6%, 10.8%, 26.1%, 34.9% and 22.6% of the ratings.

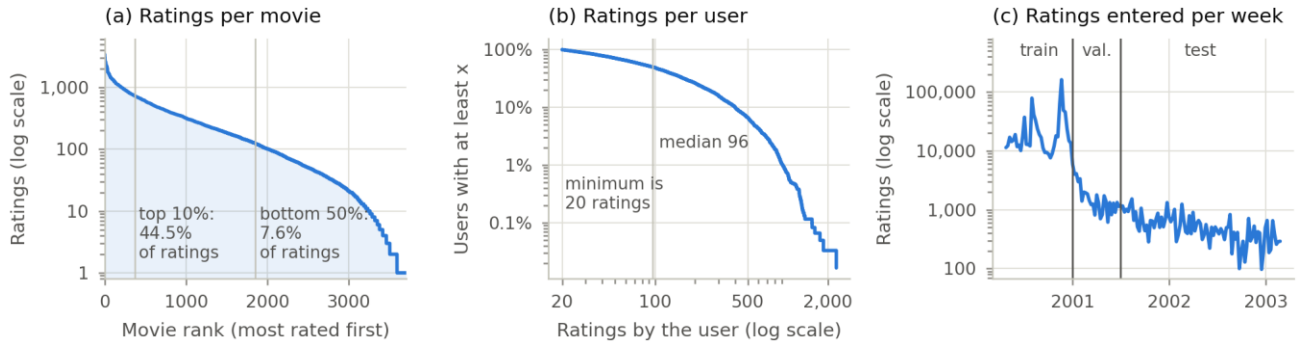


Figure 1. (a) Ratings per movie, most rated first. (b) Share of users with at least x ratings. (c) Ratings entered per week; the vertical lines are our validation and test cuts.

The long tail (Lecture 5.1, slide 5) is milder than the 80/20 rule: the 10% most-rated movies get 44.5% of the ratings, the top 20% get 65.2% and the less-rated half of the catalogue gets 7.6% (Figure 1a; Gini 0.63 over movies and 0.53 over users). Popularity also drifts. None of the ten most-rated movies of our test period is in the training top ten, eight of them are 2000 releases, and the two top-100 lists share only 54 movies.

The timestamps are the least obvious part of the data. Ratings span 25 April 2000 to 28 February 2003, but 90% were entered by 29 December 2000 (Figure 1c). Users rate in bursts when they join: 62.9% entered all their ratings within one day, and 77.4% of ratings share their exact second with another rating by the same user. A timestamp marks when a page of ratings was submitted, not when a film was watched, so the order within a session, and with it any leave-last-one-out split, is undefined. Only 799 users (13%) rate anything in our test period, and the 777 we evaluate have a median of 212 earlier ratings: the test measures heavy, returning users.

1.2 Preprocessing and feedback treatment

We parse the .dat files ourselves and check the rating range, duplicate pairs and ids; no rows are removed. The data already keeps only users with at least 20 ratings, and removing rare movies would only shrink the candidate set, so the main protocol applies no threshold (Section 1.6 shows what thresholds would have changed). Since the task is top-10 ranking (Lecture 3.1, slide 6), a held-out rating is relevant only if it is a 4 or a

5 (57.5% of all ratings); a 1 to 3 means the user saw the film and did not think much of it. Whether the models learn from all ratings as implicit interactions or only from the 4s and 5s is tuned on validation (Lecture 1.1, slide 7; Lecture 1.2, slide 11).

1.3 Train/validation/test split

We use one global temporal split (Lecture 3.1, slide 4): ratings before 1 January 2001 (UTC) train, ratings from 1 January to 30 June 2001 validate, and ratings from 1 July 2001 onwards test: 904,757 (90.5%), 47,314 (4.7%) and 48,138 (4.8%) ratings. The cuts are strict, so ratings with the same timestamp never fall on both sides. The common 80/10/10 split by count would cut on 2 December 2000, in the middle of the sign-up wave: 54% of its validation users have no earlier rating, and only 17% of its validation ratings come more than 30 days after the user's previous rating, against 92% in its test period. Validation would then measure new users and test returning ones. Our cuts put both periods in the returning-user regime: fewer than 1% of users are new, and 84% (validation) and 97% (test) of held-out ratings come more than 30 days after the user's previous one. The cost is small held-out sets: we evaluate the 892 validation and 777 test users with at least one relevant rating and some history (the test drops 19 users without a 4 or 5 and 3 without history).

1.4 Evaluation protocol

Hyperparameters are chosen by NDCG@10 on validation only; the chosen settings are refitted on training plus validation data and scored once on test (Lecture 3.1, slide 10). Each model ranks all 3,688 movies rated before 1 July 2001, minus those the user has rated, with no sampled negatives, since sampling can reorder models (slide 5; Krichene and Rendle, 2020). The 25 test ratings (0.05%) on movies nobody rated before the cut stay in the ground truth as misses, and ties are broken by a fixed random order of movies. We report Precision@10, Recall@10 (hits over the user's relevant test movies) and NDCG@10 with binary gains (slides 7-8), averaged over users, and catalogue coverage, the share of candidates in at least one top-10 list (slide 9). Our metric code reproduces the example on slide 8 and matches scikit-learn's ndcg_score on 500 random rankings to within 1e-12. Intervals are 95% bootstrap intervals over users (1,000 resamples; Efron and Tibshirani, 1993), paired comparisons add a Wilcoxon signed-rank test, and all randomness uses seed 42.

1.5 Baseline methods and settings

Most popular gives every user the same list, ranked by the number of training ratings, minus what they have seen (Lecture 1.1, slide 9). We try eight variants: all training ratings or only the last 30, 90 or 180 days, counting every rating or only the 4s and 5s. Item-based kNN (Lecture 1.2, slides 5-11) keeps for each movie i its k most similar movies with a positive similarity and scores i for a user by summing the similarities of those neighbours the user has rated (Deshpande and Karypis, 2004). We tune the similarity (cosine or Jaccard on the binary matrix, the lecture's defaults for implicit data, or adjusted cosine on the stars; Sarwar et al., 2001), the training signal for cosine and Jaccard, and k from 10 to 1,000: 35 kNN and 8 popularity settings in all.

Validation chose cosine on ratings of 4 and 5 with k = 500 (Figure 2). The training signal mattered most: using only positive ratings lifts the best cosine setting from 0.140 to 0.174 and Jaccard from 0.129 to 0.166, because otherwise a 1-star rating counts as evidence that two films belong together. NDCG@10 rises with k up to 200-500, well above the 20-50 the lecture calls typical (slide 10), but coverage falls from 18.7% at k = 10 to 10.7% at k = 500: with summed similarities, a larger neighbourhood pulls the ranking toward popular movies. Adjusted cosine peaks at k = 50. The best settings are close (Jaccard with k = 200 is 0.008 behind), so the similarity measure matters less than the signal and k. For most popular, validation chose the last 30 days of positive ratings.

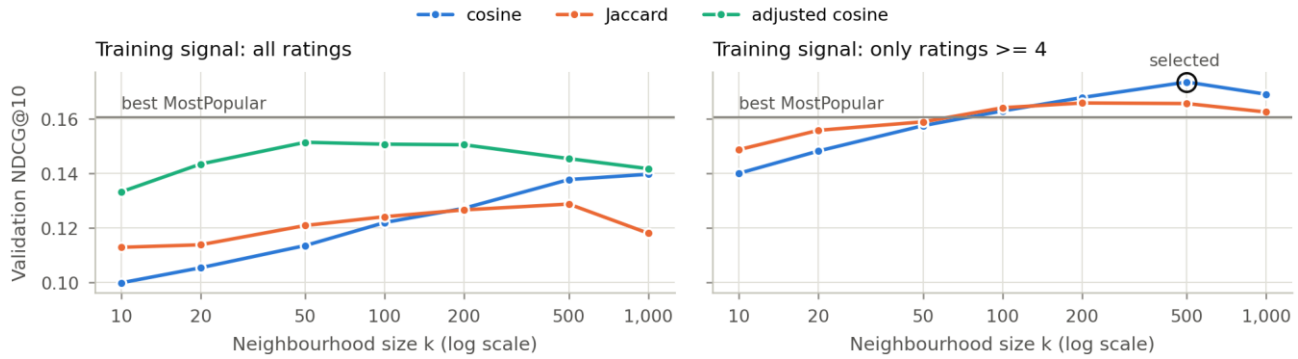


Figure 2. Validation NDCG@10 of item-based kNN against k, per similarity measure and training signal. The grey line is the best most-popular variant on validation.

1.6 Results and analysis

Table 1. Test results for 777 users and 3,688 candidate movies, with validation NDCG@10 for comparison. Bold rows are the settings chosen on validation. All 14 variants are in the results workbook.

Model and settings	Val. NDCG@10	P@10	R@10	NDCG@10 [95% CI]	Coverage
Random (seed 42)	-	0.012	0.002	0.012 [0.009, 0.015]	86.8%
Most popular: all-time, all ratings	0.135	0.153	0.057	0.169 [0.154, 0.185]	4.1%
Most popular: all-time, ratings ≥ 4	0.144	0.159	0.062	0.177 [0.162, 0.193]	4.0%
Most popular: last 30 days, ratings ≥ 4	**0.161**	**0.122**	**0.070**	**0.159 [0.145, 0.172]**	**2.0%**
Most popular: last 180 days, ratings ≥ 4	0.143	0.148	0.089	0.176 [0.161, 0.189]	2.7%
Item kNN: cosine, ratings ≥ 4, k = 500	**0.174**	**0.183**	**0.072**	**0.207 [0.192, 0.225]**	**9.8%**
Item kNN: Jaccard, ratings ≥ 4, k = 200	0.166	0.179	0.075	0.202 [0.186, 0.219]	9.8%
Item kNN: adjusted cosine, all ratings, k = 50	0.151	0.170	0.071	0.189 [0.174, 0.205]	11.8%
Item kNN: cosine, all ratings, k = 1000	0.140	0.149	0.057	0.167 [0.152, 0.181]	10.6%

Item-based kNN is the stronger baseline (Table 1). Its test NDCG@10 of 0.207 beats the selected popularity variant by 0.049 (95% CI [0.031, 0.068], Wilcoxon $p < 0.001$; better for 374 users, worse for 219) and the all-time list by 0.039 [0.028, 0.050], with 9.8% coverage against 1.6-4.1% for popularity. The gain comes from precision and ranking; against the selected popularity variant the Recall@10 difference is only 0.002 [-0.012, 0.015].

The popularity results expose a flaw we did not expect. The 30-day window that won on validation is worse on test than the all-time count of positive ratings (-0.018 [-0.036, -0.002]). A window in days is a different object at each cut: the 30 days before 1 January 2001 hold 107,641 ratings from 1,210 users, mostly new members, against 4,981 ratings from 271 users before 1 July 2001. Recent windows still have the best recall (0.064-0.089 against 0.057-0.062 for all-time counts), since they catch the 2000 releases that returning users go on to rate. A window defined by a number of ratings would mean the same at both cuts; we will test this in Part 2.

Filtering rare movies changes almost nothing. Removing movies with fewer than 5, 10, 20 or 50 training ratings deletes 0.06% to 2.4% of training rows and leaves as few as 2,465 candidates, yet moves NDCG@10 by less than 0.002, even when their test ratings are dropped too: neither baseline recommends these movies. Evaluating only users with 50 or more training ratings matters more: kNN loses 0.002, popularity gains 0.008 and the gap falls from 0.049 to 0.039. No threshold moved the results by more than the gap, but the user threshold moves the gap itself, a hint that the baselines serve light and heavy users differently (to be broken down in Part 2).

The split matters most. With the 80/10/10 split, validation chose nearly the same settings (k = 1000; a 90-day window), and they score within 0.001 of ours on our test set. On that split's own test set, however, kNN reaches 0.246 and popularity 0.211. Moving the cut shifted the numbers by 0.04-0.05, as much as the gap

between the baselines, while their order held. Results from different splits, including other teams', are not directly comparable.

2. Part 2 - Models and Honest Comparison

2.1 Models

2.2 Hyperparameter tuning and variability

2.3 Overall results

2.4 Disaggregated analysis

2.5 Sparsity ablation

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

3.2 Method

3.3 Results and trade-offs

3.4 Cost and interpretation

3.5 "One number we cannot explain"

4. Discussion and Conclusions

5. Contribution Statement

Anas Uddin: set up the repository and the evaluation code, ran the Part 1 experiments and drafted Sections 1.1-1.6. Anas also worked through the requirements and shaped the solution together with Melek and Menaka.

Melek Chelik: took an equal share of the implementation. With the team, Melek worked through the Part 1 requirements and helped derive and steer the solution, including the protocol and the baselines.

Menaka Wellane Hewage: contributed equally to the implementation. Menaka took part in the team's discussions of the requirements and helped brainstorm, work out and guide the solution with us.

6. Use of AI Assistants

We used Claude (Anthropic) in the Claude Code desktop app to plan the repository, write first versions of the code and unit tests, run the experiments and draft the Part 1 text from the generated result tables. We read and tested the code, checked the reported numbers against results/results.csv and results/tables, and edited the text; the team is responsible for all of it.

References

Deshpande, M. and Karypis, G. (2004). Item-based top-N recommendation algorithms. ACM Transactions on Information Systems, 22(1), 143-177.

Efron, B. and Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman and Hall.

Harper, F. M. and Konstan, J. A. (2015). The MovieLens datasets: History and context. ACM Transactions on Interactive Intelligent Systems, 5(4), Article 19.

Krichene, W. and Rendle, S. (2020). On sampled metrics for item recommendation. In Proceedings of KDD 2020, 1748-1757.

Sarwar, B., Karypis, G., Konstan, J. and Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of WWW 2001, 285-295.

Stefanidis, K. (2026). DATA.ML.360 Recommender Systems, lecture slides 1.1-6.2. Tampere University.